

ICS-PA2-1 实验报告

241220029 杨智宇

思考题

(一) 使用 `hexdump` 命令查看测试用例的文件，所显示的文件的内容对应模拟内存的哪一个部分？指令在机器中表示的形式是什么？

答：.img 文件的内容对应模拟内存的从 0x30000 开始的一段空间，如下图所示：



由于 PA2-1 还没有实现装载 ELF 文件的功能，框架代码直接将文件拷贝到模拟内存作为测试样例，位于上图 Testcase Binary 所示区域。

另外指令在机器中是以二进制形式表示的，每条指令由一或多个字节组成，分为 opcode 和 operands。

(二) 如果去掉 `instr_execute_2op()` 函数前面的 `static` 关键字会发生什么情况？为什么？

答：如果只有一个 `instr_execute_2op()` 函数前面缺少 `static` 关键字，则对框架代码的运行没有影响；但是一旦有两个及以上的 `instr_execute_2op()` 函数前面缺少 `static` 关键字，则会在编译时发生重定义错误，如下图报错：

```
/usr/bin/ld: src/cpu/instr/add.o: in function 'instr_execute_2op':  
/home/pa241220029/pa_nju/nemu/src/cpu/instr/add.o:4: multiple definition of 'instr_execute_2op'; src/cpu/instr/add.o:/home/pa241220029/pa_nju/nemu/src/cpu/instr/add.o:4: first defined here  
collect2: error: ld returned 1 exit status
```

由于 `static` 关键字的作用是让某个函数只可被定义它的文件内的其他函数调用，但不能被其他文件的函数调用。因此可以推测出现上述现象的原因如下：如果只有一个 `instr_execute_2op()` 函数前面缺少 `static` 关键字，编译时其他 `instr_execute_2op()` 函数都可以正确链接，而没有加 `static` 关键字的函数为全局函数，由于其文件中没有重定义，也可以正确运行；如果有两个及以上的 `instr_execute_2op()` 函数前面缺少 `static` 关键字，例如 `nemu/src/cpu/instr/add.c` 和 `nemu/src/cpu/instr/and.c` 中都定义了同名的全局函数 `instr_execute_2op()`，那么链接的时候就会产生重定义错误。

(三) 为什么 `test-float` 会 fail？以后在写和浮点数相关的程序的时候要注意什么？

答：我们首先来关注一下 test-float 的源代码：

```
test-float.c
1 #include "trap.h"
2
3 int main()
4 {
5
6     float a = 1.2, b = 1;
7     float c = a + b;
8     if (c == 2.2)
9     ;
10    else
11        HIT_BAD_TRAP;
12    c = a * b;
13    if (c == 1.2)
14    ;
15    else
16        HIT_BAD_TRAP;
17
18    c = a / b;
19    if (c == 1.2)
20    ;
21    else
22        HIT_BAD_TRAP;
23
24    c = a - b;
25    if (c == 0.2) // this will fail, and also fails for native program, interesting, can be used as a quiz
26    ;
27    else
28        HIT_BAD_TRAP;
29
30    HIT_GOOD_TRAP;
31    return 0;
32 }
```

我推测可能有以下两种原因：

(1) 测试用例中 1.2⁽¹⁰⁾ 表示成 IEEE754 规定的浮点数时，尾数是 1.001100110011……⁽²⁾，可见这不是精确的表达，即十进制转二进制存在一些误差。因此使用“==”来判断浮点运算可能会产生错误。

(2) 我们对测试用例 test-float 反汇编，截取如下图的一部分观察：

300b0:	80 e4 45	and \$0x45,%ah
300b3:	80 fc 40	cmp \$0x40,%ah
300b6:	74 06	je 300be <main+0xb9>
300b8:	b8 01 00 00 00	mov \$0x1,%eax
300bd:	82	nemu_trap
300be:	b8 00 00 00 00	mov \$0x0,%eax
300c3:	82	nemu_trap
300c4:	b8 00 00 00 00	mov \$0x0,%eax
300c9:	c9	leave
300ca:	c3	ret

可见 and、cmp 之类的运算指令的 opcode 第一位是 0x80，也就是 group_1_b 中的各个指令，这些指令调用的是 ALU 进行整数运算，而不是浮点运算应该调用的 FPU，这也许是得到错误结果的原因。

我们以后写浮点数相关的程序时，一方面一定要注意需要使用正确的浮点运算指令；另一方面不要忽视计算机中能表示的浮点数并非连续，因此运算中会产生误差，因此，对于浮点数 a, b 进行比较时，常采用 $a-e < b < a+e$ （其中 e 为一个很小的正数）即认为 $a == b$ 。